Commissioning and test roadmap — ASKET ASV

Who this is for: the engineer who will test this boat's hardware against its software before it goes in the water. It assumes you know electronics and machinery. It does **not** assume you have seen this project, ROS 2, Linux containers or git before. Where something is not general engineering knowledge, it is explained, and the reason is given as well as the command — a command you do not understand is a command you cannot debug at the water's edge.

What this document cannot do: nobody has had access to the Jetson while writing it. Every command here was checked to exist in the repository — the script, launch file, service, topic or path is real — but **the commands have not been run on the boat**. Where a value, a name or a behaviour is unknown, this document says UNKNOWN instead of guessing. See section 6.

Read this first

One page. Then start.

Who reads what

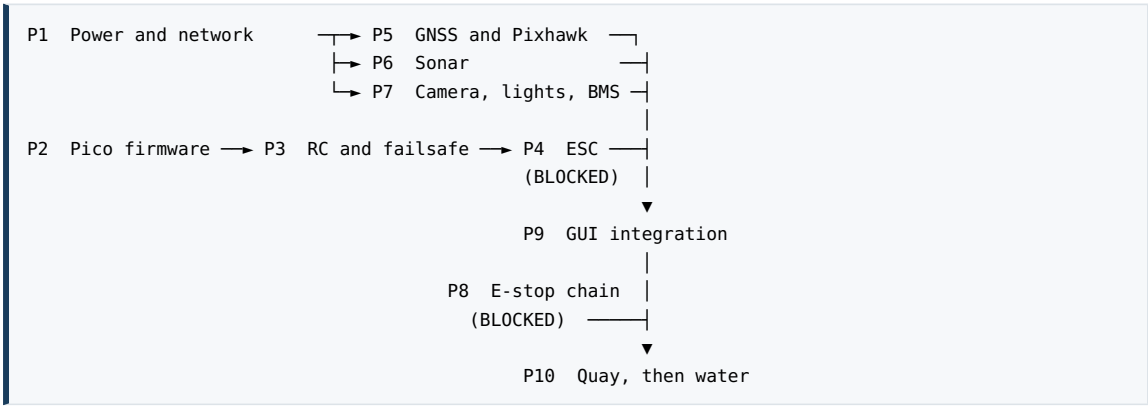
You are	Read
Testing the hardware — the main reader	this whole document, in order
Operating the boat on the day	`docs/OPERATOR_CHECKLIST.md`, one page
Working on the GUI	`docs/GUI_DEVELOPER_GUIDE.md`
Just need the sonar	`docs/SONAR_BRINGUP.md` — the only part of the boat verified in water
Checking what is actually fitted	`docs/HARDWARE_DECISIONS.md` — the reference. If it and this document disagree, it wins

The three sections to read before you touch anything

1. **Section 0** — what is verified and what is not. It is mostly bad news, stated plainly, and it will save you from testing something that was never going to work.

2. **Section 1** — the safety rules. Two of them are not obvious and both come from measurement, not principle: the ESC bench-supply current limits, and the fact that **the sonar keeps pinging after you close the software**.
3. **Section 2** — SSH, containers, ROS, git. Skip what you know. Do not skip **2.3**: on the production container a `git checkout` changes nothing, and the published image is from October and has none of the sonar work in it. That trap catches everybody once.

The order of the phases



P1 first, always. P2 before P3, and P3 before P4 — each is the previous one's safety net. P5, P6 and P7 are independent of that chain and of each other, so do them whenever the hardware and the weather allow. P9 wants P1, P2 and P5 at least. **P10 needs every one of them, P8 included.**

What is blocked today

Phase	Blocked by	Who unblocks it
P4 — ESC and motors	The **relay on GPIO21 is not wired** (so the Pico cannot cut ESC power at all) and the **physical E-stop button is planned, not fitted**	whoever wires and tests both. A hardware job, not a software one
P8 — E-stop chain	The same two, plus the **GPIO20 feedback divider**, also not wired	same
P10 — water trials	P4 and P8	same

Everything else can be done now. **P2 is the most valuable thing you can do first**: the firmware believed to be on the Pico is v4, the stack drives v5 only, and until that is settled nothing about the motor path means anything.

Two things that will bite you

- **The sonar keeps pinging when you close the software.** Stop it explicitly, while the bridge is still running, before the transducer leaves the water. Pinging in air can damage it. See 1.6.
- **There is no obstacle detection on this boat.** None. The navigation stack knows a geofence and a speed limit and nothing about what is in the water. The person holding the transmitter is the obstacle avoidance.

As you go

Fill in the results sheet while you work, not afterwards, and **change the labels in this document when you verify something** — with the date and your name. That is the most valuable thing you will leave behind.

How facts are labelled

Every technical claim carries one of these. There is no "should work".

Label	Meaning
VERIFIED	Measured on the real hardware. Date and person given.
TESTED IN SOFTWARE ONLY	Covered by unit tests, host-side firmware tests or a simulator. No hardware involved.
NOT VERIFIED	Written and plausible; nothing has tested it, in software or on hardware.
UNKNOWN	Nobody knows. Listed in section 6.

If you verify something, **change the label and write the date and your name.** That is the single most valuable thing you can leave behind.

---

0. Project status

The boat: ASKET, an autonomous surface vessel for a seabed survey in Namibia for Kelp Blue. Two thrusters, a Raspberry Pi Pico for the low-level control, a Pixhawk flight controller for position and heading, a Jetson AGX Orin running ROS 2 and the mission GUI, and a Cerulean Omniscan 3D sonar.

Source of truth for all of this: `docs/HARDWARE_DECISIONS.md`. If that file and this one disagree, that file wins and this one is the bug.

0.1 Hardware

Item	State	Label
Jetson AGX Orin, ROS 2 Jazzy in a container	Present and running. Logs in as `node`, hostname `node2026`	**VERIFIED 2026-10-28** (seen in the terminal)
MikroTik CSS610 switch, NetMetal 5 ax (boat), mANTBox ax 15s (shore)	Present; nothing in the software reads the radios	NOT VERIFIED as a complete link
Raspberry Pi Pico 2, USB serial	Present. **v4 is declared flashed** (Auxence, not checked against the hardware). The stack drives **v5 only** — so if the declaration is right, the motor path is blocked until v5 is flashed	**NOT VERIFIED** (declared v4)
`pico-node_v5` firmware	Exists in the repository; **never compiled by the Arduino toolchain, never run on a Pico**. `docs/BENCH_TEST_PHASE_B.md` §0 step 7 calls itself the first such build	TESTED IN SOFTWARE ONLY (host stub, all tests pass)
2× Blue Robotics T200 + 2× Basic ESC 30A, PWM from Pico GPIO15/16	Present	NOT VERIFIED
RP4TD ExpressLRS receiver + Radiomaster MT12, SBUS into Pico GPIO5	Present. SBUS output mode and what it does on radio loss are unconfirmed	NOT VERIFIED
ESC relay (A3KM) on GPIO21	**NOT WIRED.** Without it the Pico cannot cut ESC power at all	VERIFIED absent
E-stop button feedback on GPIO20	**NOT WIRED**; firmware path compiled out (`ESTOP_FEEDBACK_ENABLED 0`)	VERIFIED absent
Physical E-stop button in the ESC power line	**PLANNED, not fitted.** Blocks P4 together with the relay	declared 2026-11-10 (Auxence)
3 lights + buzzer, GPIO10/11/12/13	Present	NOT VERIFIED on the loom
Pixhawk 4	Present. **PX4 or ArduPilot is unconfirmed** — the code assumes ArduPilot Rover by default	UNKNOWN
Holybro H-RTK UM982 GNSS, dual antenna, on the Pixhawk	Present. Dual-antenna heading configuration not done	NOT VERIFIED
F9P RTK base ashore, corrections to the boat	Not implemented in software at all	NOT VERIFIED
2× BMS BQ76920 via MCP2221	**Only one is wired.** `enable_bms2` defaults to false	VERIFIED by the configuration
2× LiPo 4S 8000 mAh in parallel, ≈237 Wh	Present	VERIFIED by the configuration
Cerulean Omniscan 3D 450 SS Modular	Present and working	**VERIFIED in water 2026-11-06** (Salvador Cantuarias Brañes) — see 0.3
OAK-D-LR camera over PoE	**No driver installed.** The GUI reads `/oak/rgb/image_raw/compressed`, a name nobody has confirmed	UNKNOWN
Lidar (RPLidar / Livox)	**Not on the boat.** So the boat has **no obstacle detection of any kind**	VERIFIED absent

0.2 Software

Item	State	Label
ROS 2 nodes: `pico_bridge`, `mavros`, `imu_gps_driver`, EKF, Nav2, `mission_manager`, `boat_bt`	Present, launched by `src/bringup/launch/njord.launch.py`	TESTED IN SOFTWARE ONLY
`omniscan_bridge` (sonar → ROS 2)	Works against the real sonar	**VERIFIED 2026-11-06**
`gui_backend` + the React GUI, port 8090	Builds and runs; never run against the real boat end to end	TESTED IN SOFTWARE ONLY
Pre-flight (`system_test`), 21 checks, 11 of them critical	Logic tested; never run against real hardware	TESTED IN SOFTWARE ONLY
`mission_recorder`	Tested in software	TESTED IN SOFTWARE ONLY
Nav2 survey profile: geofence keep-out + speed cap, **no obstacle layers**	Measured in a Nav2 Jazzy image with a synthetic fence: all nine nodes active, a cell beyond the fence lethal, a plan outside refused, 2 m/s in comes out as 0.80 m/s	TESTED IN SOFTWARE ONLY
Firmware host tests (`make -C firmware/test`)	**ALL PASS, 0 failures** (run 2026-11-09)	TESTED IN SOFTWARE ONLY
Python test suite (`pytest`)	See section 0.4	TESTED IN SOFTWARE ONLY
`sonar_bridge_node` in `src/comms/cloud_bridge/`	Parses message 2198 with a layout taken from prose, **never checked against a device**. Off by default (`enable_sonar_bridge:=false`). Do not trust its output	NOT VERIFIED — and its own source file says so

0.3 The sonar, which is the one thing that is properly verified

VERIFIED 2026-11-06, Salvador Cantuarias Brañes, transducer facing up in a shallow bucket under about 10 cm of water:

- The sonar answers on **192.168.2.21**, data port **TCP 62312** (not UDP, and not the 51200 Cerulean's documentation names — that port is closed).
- The stream **is** Ping Protocol: 1628 frames in 75 s, 0 invalid, 0 bytes discarded.
- `OS3D_POINT_SET` decodes: 388 point sets, 0 length mismatches, 0 decode errors.
- **Only one TCP client at a time.** A second connection is refused.
- **It keeps pinging with no client connected** — 5.0 Hz throughout about 14 minutes that included the bridge being stopped.
- `~/start_pinging` starts it; `~/stop_pinging` stops it.
- Its clock is **not set**, so timestamps read as 1970 and `/diagnostics` sits at error level. Expected until the NTP step in `docs/SETUP.md`.

Still UNKNOWN: whether the points are in the **right place** (nothing measured against a known target), whether `pt_type` ever reports bottom (everything in the bucket was unclassified), whether SonarView holds the port the way a plain client does, and all of the above **on the Jetson** — the 2026-11-06 session ran on a Windows laptop through a dev container.

Full detail: `docs/SONAR_BRINGUP.md`.

0.4 Test results, 2026-11-09

Run in this repository on branch `integration-pre-sea-trial`, on a Linux container with Python 3.11, **no ROS 2 and no hardware**.

Suite	Command	Result
Firmware host tests	`make -C firmware/test`	**ALL PASS, 0 failures.** Two binaries: `fwtest` as shipped, `fwtest_feedback` with the GPIO20 path enabled
Python tests	`pytest`	Before `npm install`: **4 errors**, everything else passing. After `npm install`: **1225 passed, 6 skipped, 0 failed, 0 errors** in 181 s
Linter	`python3 -m flake8 --max-line-length=100 src/`	**168 findings, every one pre-existing.** Mostly `E501` line-length (80) and `E221` whitespace-before-operator (59), concentrated in the older club packages: `calibration` 44, `perception` 37, `bringup` 25, `comms` 14, `control` 12, `sensors` 10. The GUI and pre-flight packages account for 6 between them. No new findings on this branch — it changes only Markdown
Frontend build	`cd src/asket_gui && npm install && npm run build`	**Succeeds**, 81 modules, output written to `src/gui_backend/gui_backend/static/` as designed

The 4 errors, explained, because they will reappear on a fresh clone: all four are in `src/gui_backend/test/test_gui_alarm_sound.py`, which runs the frontend's JavaScript under Node to check that amber alarms stay silent and red ones beep. Without `node_modules` it fails at `ERR_MODULE_NOT_FOUND: Cannot find package 'react'`. **The fix is `cd src/asket_gui && npm install`**, not a code change. They are an environment problem and have been there since before this branch.

The 6 skips are not failures. They are tests that need `rclpy` — a real ROS 2 install — and are skipped automatically without one. Three of them matter more than the number suggests, because they cover the three bugs the 2026-11-06 water test exposed:

Skipped test	Covers
`src/omniscan_bridge/test/test_steady_timers.py`	the timer fix: on the ROS clock, timers followed system-clock steps and stalled for up to 27 s, overflowing the receive queue and costing 39 % of pings
`src/omniscan_bridge/test/test_status_message.py`	the `SonarStatus.gain_setting` type fix: auto gain is −1, which crashed the node at its first status publish on an idle sonar

Skipped test	Covers
`src/omniscan_bridge/test/test_idle_diagnostics.py`	what the diagnostics say about an idle sonar

So on this machine those three fixes are unverified. Run `colcon test` on the Jetson, where `rclpy` exists, and record the result. This is the one gap in the software testing that a tester can close without touching the boat.

What `pytest` does not cover at all: the competition packages (`src/sensors/`, `src/control/`, `src/mission/`, `src/perception/`, …). `pytest.ini` scopes the run to the GUI and sonar packages on purpose, because the others import `rclpy` and the ROS message packages — sweeping them into a bare `pytest` would make the GUI's "runs with no ROS installed" promise fail on a laptop. **Those are run by `colcon test`, which needs a ROS 2 install, and has not been run here.** Both, always.

---

1. Safety rules — non-negotiable

Read all of these before you power anything. They are not advice.

1.1 The hardware wins

The hardware killswitch and RC channel 8 cut propulsion independently of all software. Nothing in the Jetson, ROS 2, the network, the GUI or your browser can override, delay or interfere with them. This is the design (`docs/safety.md`), and it is why the rest of this document can be relaxed about software crashing.

The consequence for you: **the transmitter is a safety device, not a test instrument.** One person holds it, has nothing else to do, and is watching the boat.

1.2 What the software calls a stop is not an emergency stop

The GUI has a button labelled **"Cut propulsion"**. It is a soft latch. It is deliberately not called "Emergency stop", so that nobody comes to rely on it as one. If you need the motors to stop now, use Ch8 or the killswitch.

1.3 Before any powered test

☐

- [ ] **Killswitch / E-stop button within reach** of the person at the laptop.
- [ ] **Thrusters clear**: no hands, cables, tools or floor within reach of the blades.
- [ ] **One person on the transmitter**, transmitter on, doing nothing else.
- [ ] **Ch8 down** (E-stop zone) before power is applied. It moves up only when a step says so.
- [ ] **ESC battery disconnected** unless the step is specifically a powered-ESC step.
- [ ] **One test at a time.** Do not combine phases. When two things change and something breaks, you have learned nothing.

1.4 Electrical limits — ESCs on a bench supply first

The first time the ESCs are powered, use a current-limited lab supply rather than the LiPo. A wiring mistake then costs a current-limit beep instead of a fire. Values from `docs/BENCH_TEST_PHASE_B.md` §2a:

Setting	Value	Why
Voltage	**14.8 V** (4S nominal)	inside the T200's and the ESCs' range, below a full pack (16.8 V)
Current limit, power-up and arming	**2 A**	the ESCs' start-up tones draw well under 1 A; 2 A rides through their capacitor inrush
Current limit, low-throttle spins	**5 A**	two T200 at 10–15 % throttle draw about 1–3 A together in air

Keep your hand on the supply's output switch.

If the supply folds back at switch-on and the ESC tones loop, the **inrush** is tripping it — the brief surge as the ESCs' capacitors charge, which is much larger than their running current. Raise the limit to 3 A **for the power-up only**, then put it back.

A T200 can draw over 20 A at full throttle. On a 5 A supply it will current-limit and the ESC will brown out mid-test, which looks like a software fault and is not. Stay at low throttle.

1.5 Thrusters out of the water

T200 thrusters are **water-lubricated**. Out of the water, spin them **only in short bursts of a few seconds at low throttle**. Running them dry damages them.

1.6 The sonar

- **Never ping in air.** The transducer can be damaged. It goes under water before it is told to ping, and it stops pinging before it comes out.
- **Closing the software does not stop it.** VERIFIED 2026-11-06: the sonar kept pinging at 5.0 Hz through 14 minutes that included the bridge being stopped. Stopping it is an explicit service call, and it must be made **while the bridge is still running**, because the service goes away with it:

  ```
  ros2 service call /omniscan_bridge/stop_pinging std_srvs/srv/Trigger
  ```

  Watch that the points stop. **If they do not, cut the sonar's power.**
- Nothing in this repository starts the sonar pinging by itself. The bridge's `send_ping_parameters` parameter is `false` by default, deliberately.

1.7 There is no obstacle detection

No lidar, no working camera pipeline. Nav2's survey profile knows two things: where the boat may go (a geofence) and how fast (0.8 m/s). **It does not know about anything in the water.** Autonomous operation needs a human watching, a clear area, and the transmitter in hand.

---

2. The basics

Skip any part of this you already know. Nothing here is specific to this boat except the addresses and the paths.

2.1 The network

What is on it

The `10.10.20.x` plan has never been built. Nothing has been observed on that subnet. Read the "Actual" column, not the "Planned" one.

Device	Planned address	Actually seen	Label
Jetson, wired (`eth0`)	10.10.20.10	**192.168.88.10/24** — the MikroTik switch's own factory subnet	**VERIFIED 2026-10-28**
Jetson, Wi-Fi (`wlan0`)	—	**DHCP; 192.168.1.236 that day.** It changes — look it up each time, do not write it down	**VERIFIED 2026-10-28**
Jetson, temporary	—	**192.168.2.1/24**, added by hand to reach the sonar. Gone on reboot	**VERIFIED 2026-10-28**
Omniscan 3D sonar	10.10.20.31	**192.168.2.21**	**VERIFIED 2026-11-06**
Sonar's two internal boards	—	**192.168.2.201**, **192.168.2.202**, UDP 51221/51222 with the host	VERIFIED 2026-10-28
MikroTik CSS610 switch	10.10.20.2	not confirmed; the `192.168.88.x` addressing suggests it is still on its factory default (MikroTik ships `192.168.88.1`)	NOT VERIFIED
MikroTik NetMetal 5 ax (boat radio)	10.10.20.3	—	NOT VERIFIED
MikroTik mANTBox ax 15s (shore radio)	10.10.20.4	—	NOT VERIFIED
OAK-D-LR camera	10.10.20.21	no driver, never addressed	NOT VERIFIED
Livox lidar (reserved)	10.10.20.22	not fitted	—
Shore laptops	10.10.20.100–199, DHCP	—	NOT VERIFIED

So **three** subnets were live on the Jetson on 2026-10-28 — `192.168.88.x` wired, `192.168.1.x` over Wi-Fi, and the hand-added `192.168.2.x` for the sonar — and **none of them is the plan**. Whoever builds `10.10.20.0/24` will have to re-address the switch, the two radios and the Jetson, and then decide what happens to the sonar (below).

The GUI is at `http://10.10.20.10:8090` **once the plan exists**. Until then it is on whichever address the Jetson actually has: `ip addr show` on the Jetson, then `http://<that address>:8090`.

The conflict you need to know about

The plan is one flat `10.10.20.0/24` subnet bridged over the Wi-Fi link. **The sonar is not on it.** It answers on `192.168.2.21`, and SonarView makes its two internal boards talk to the host on `192.168.2.x` with the host on `.1`.

And `.21` in the plan is already the OAK-D-LR.

> **Conflit IP :** OAK-D-LR prévue en .21 dans le plan 10.10.20.x ; le sonar est en 192.168.2.21 par défaut et SonarView le fait dialoguer sur 192.168.2.x avec l'hôte en .1. Décision réseau à prendre.

This decision has not been made, and this document does not make it. Three things are open at once: which subnet the sonar ends up on, who gets `.21`, and whether the boards' own `192.168.2.201/.202` traffic survives a re-address — nobody has checked whether those follow the sonar or are set separately.

Until it is decided, the software follows the measurement: the sonar default is `192.168.2.21` everywhere, overridable with the `ASKET_SONAR_HOST` environment variable, and the `10.10.20.x` plan is untouched.

Do not re-address the sonar on the day of a survey. If it goes wrong you have no sonar and no way back without SonarView and a wired laptop.

Giving the host a temporary address on the sonar's network

The sonar is on a different subnet from everything else, so the Jetson needs a second address to reach it. On the Jetson:

```
ip addr show                                  # find the wired interface's name
sudo ip addr add 192.168.2.1/24 dev eth0      # ADD, not replace
ip addr show eth0                             # check it took
```

Replace `eth0` with the real name — `enP8p1s0` and similar are common on Jetsons.

Why this is safe: `ip addr add` adds an address to an interface that can hold several. Your Wi-Fi, any hotspot and your SSH session are on different interfaces and are untouched.

It does not persist. Nothing is written to disk, so **a reboot undoes it**. That is a feature: a mistake here cannot survive a power cycle. To undo it immediately:

```
sudo ip addr del 192.168.2.1/24 dev eth0
```

On a **Windows** laptop instead (Administrator PowerShell, this one does persist):

```
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.2.1 -PrefixLength 24
# to undo:
Remove-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.2.1
Set-NetIPInterface -InterfaceAlias "Ethernet" -Dhcp Enabled
```

2.2 Getting onto the Jetson

The Jetson has no screen you will want to use. You work on it over **SSH**: a remote terminal, from your laptop, over the network.

```
ssh node@node2026
```

The user is **`node`** and the hostname is **`node2026`** — VERIFIED 2026-10-28, read off the Jetson's own terminal prompt. If the name does not resolve on your network, use the address instead:

```
ssh node@192.168.88.10     # the wired address seen on 2026-10-28
ssh node@<wifi address>    # DHCP, changes; ask, or look at the Jetson's screen
```

Ignore the `ssh pi@boat.local` example in `README.md` — that is the older Raspberry Pi deployment, not this Jetson.

Copying files **to** the Jetson, and **from** it:

```
scp ./localfile  node@node2026:/home/node/                # to
scp node@node2026:/srv/asket/data/missions/* ./           # from
scp -r node@node2026:/srv/asket/data/missions ./          # a whole directory
```

Two SSH sessions at once is normal and useful: one to run things, one to watch logs.

2.3 Containers — and where the code actually lives

The ROS 2 software does not run directly on the Jetson. It runs in a **container**: a packaged filesystem with its own installed software, isolated from the host. The Jetson uses **Podman**, which behaves like Docker; if you know `docker`, every command below works the same with `docker` in place of `podman`.

The commands you need

```
sudo podman ps                                  # what is running
sudo podman ps --format '{{.Names}}\t{{.Image}}'  # names and images only
sudo podman logs asket                          # the container's output
sudo podman logs -f asket                       # ...and follow it live
sudo podman exec -it asket bash                 # a shell INSIDE the container
sudo podman inspect asket --format '{{range .Mounts}}{{.Source}} -> {{.Destination}}{{"\n"}}{{end}}'
```

The stack normally runs as a **systemd service** called `asket.service`, so:

```
sudo systemctl status asket.service
sudo systemctl restart asket.service
sudo systemctl stop asket.service
journalctl -u asket.service -f                  # its log, live
```

The trap: where does the code come from?

This catches everybody once. There are two different container setups in this repository and the answer is opposite in each.

	Production	**Development**
Defined by	`Containerfile`, target `prod`	`.devcontainer/`
Started by	`asket.service` (Podman)	VS Code, "Reopen in Container"
The code is	**copied into the image** and compiled to `/opt/njord`	**mounted** from your checkout at `/workspace`
Repository mounted?	**No** — only `/dev`, `/run/udev` and `/data`	Yes
`/opt/njord` exists?	Yes	No
After `git checkout`	**the image must be rebuilt**	`colcon build` is enough

Find out which you are in — from the host:

```
sudo podman inspect asket --format '{{range .Mounts}}{{.Source}} -> {{.Destination}}{{"\n"}}{{end}}'
```

Only `/dev`, `/run/udev`, `/srv/asket/data` → production, code from the image. Your repository mounted on `/workspace` → development.

Or from inside the container:

```
ls -d /opt/njord 2>/dev/null       && echo "=> CODE FROM THE IMAGE"
ls -d /workspace/install 2>/dev/null && echo "=> CODE FROM THE MOUNTED REPOSITORY"
```

The second trap: the published image is old

`asket.service` ships with:

```
Environment=IMAGE=ghcr.io/node-engineering-club/node-ros-2026:latest
```

That image is built by CI **only from `main`** (`.github/workflows/build.yaml`, `branches: [main]`), and `main` is from 2026-10-22. **It does not contain the sonar work or the v5 firmware support.** A `podman pull` will not help you. You must build the image from the branch.

Building and deploying the image, on the Jetson

```
cd ~/node-ros-namibia                    # wherever the repository lives
git fetch origin
git checkout integration-pre-sea-trial
git pull
git submodule update --init --recursive  # src/sllidar_ros2; COPY only copies what is on disk
sudo podman build --target prod -t localhost/asket:pre-sea-trial .
```

This compiles the whole ROS workspace **and** the web frontend inside the image. **It takes a long time on a Jetson.** Start it when you do not need the boat. You do not need to run `npm` yourself: the image build does it in its own stage.

Then point the service at it:

```
sudo sed -i 's|^Environment=IMAGE=.*|Environment=IMAGE=localhost/asket:pre-sea-trial|' \
  /etc/systemd/system/asket.service
sudo systemctl daemon-reload
sudo systemctl restart asket.service
journalctl -u asket.service -f
```

Verify you are running the new code — do not assume:

```
sudo podman exec asket grep -c stop_pinging \
  /opt/njord/lib/python3.12/site-packages/omniscan_bridge/omniscan_bridge_node.py
```

Greater than zero means you have the 2026-11-06 sonar work. Zero, or a "no such file" error, means you are still on the old image.

What survives a restart

The container runs with `--rm`: it is **thrown away** every time it stops. Anything written inside it is gone. What must be kept is written under `/data` in the container, which is `/srv/asket/data` on the Jetson's own disk:

Kept on the host at	Written by	Path in the container
`/srv/asket/data/missions/`	`mission_recorder`	`/data/missions/`
`/srv/asket/data/system_test.jsonl`	`system_test`	`/data/system_test.jsonl`
`/srv/asket/data/maps/`	`gui_backend` (map tiles)	`/data/maps/`
`/srv/asket/data/setup/asket_setup.yaml`	the GUI's Setup page	`/data/setup/asket_setup.yaml`

ROS logs are **not** kept. Full detail: `deploy/jetson/README.md`.

Never two stacks at once

There is an older Raspberry Pi installer (`scripts/init.sh`) that installs `njord.service`, running the same launch file in a container named `njord`. If both ran you would get two `pico_bridge` fighting over `/dev/pico`, every topic published twice and two GUIs on port 8090. Check:

```
systemctl is-active asket.service njord.service
sudo podman ps --format '{{.Names}}'          # expect only "asket"
```

If `njord.service` is installed:
`sudo systemctl disable --now njord.service njord-update.service`.

> **Security note:** `scripts/init.sh` contains a hard-coded container-registry token. It is a known issue the team is handling — **it must be revoked and removed from the git history.** Do not copy that file anywhere, do not paste its contents into a ticket, and do not use it on the Jetson.

2.4 ROS 2, the minimum you need

ROS 2 is the framework the boat's software is built on. Three ideas:

- **Nodes** are programs. One per job: `pico_bridge` talks to the Pico, `mavros` talks to the Pixhawk.
- **Topics** are named streams nodes publish and subscribe to, like `/pico/status`. Many readers, many writers, no reply.

- **Services** are request/response calls, like `/omniscan_bridge/stop_pinging`. One caller, one answer.

"Sourcing" the environment. Before any `ros2` command works, your shell has to be told where ROS and this workspace are installed. That is what `source` does: it runs a script that sets environment variables in your current shell. It applies to **that shell only** — every new terminal needs it again, and so does every `podman exec`.

In the production container:

```
sudo podman exec -it asket bash
source /opt/ros/jazzy/setup.bash
source /opt/njord/setup.bash
```

In a development container, the second line is `source /workspace/install/setup.bash`.

If you get `ros2: command not found`, you have not sourced. If `ros2 topic list` shows only `/parameter_events` and `/rosout`, you sourced ROS but not the workspace.

The commands you will actually use:

```
ros2 node list                              # which programs are running
ros2 topic list                             # what streams exist
ros2 topic echo /pico/status --field data   # watch one stream's contents
ros2 topic echo /diagnostics --once         # one message, then stop
ros2 topic hz /mavros/imu/data              # how fast a stream is arriving
ros2 service list                           # what can be called
ros2 service list -t                        # ...with each one's type
ros2 param get /omniscan_bridge host        # read a node's setting
```

Calling a service needs its **type**, which `ros2 service list -t` gives you. Two you will use:

```
# the sonar's two Trigger services: no arguments
ros2 service call /omniscan_bridge/stop_pinging std_srvs/srv/Trigger

# the pre-flight: a custom type, and an empty `only` means every passive check
ros2 service call /system_test/run asket_interfaces/srv/RunSystemTest \
  "{only: [], include_active: false, consent_token: ''}"
```

`include_active` is **refused outright** by this service, so there is no path through it that can reach a thruster — the active motor tests are deliberately not wired to a service at all (`src/system_test/README.md`). Pass `false`. Motor testing is Phase P4, by hand, with somebody standing next to the boat.

`ros2 topic echo` prints until you stop it with **Ctrl-C**. That is normal.

Reading diagnostics. `/diagnostics` is where nodes report their own health as named key/value pairs. It is more informative than any panel, and it is the first place to look when the GUI says something vague:

```
ros2 topic echo /diagnostics --once
```

2.5 Git, the minimum you need

Git tracks versions of the code. A **branch** is one line of development.

Use `integration-pre-sea-trial`. It contains everything: the hardware alignment work, the sonar work verified on 2026-11-06, and the GUI work. The older branches `hw-alignment` and `GUI_AUXI` are both missing the sonar fixes, and `hw-alignment` additionally crashes about a second after starting against an idle sonar.

```
cd ~/node-ros-namibia
git fetch origin                          # get the latest from the server
git checkout integration-pre-sea-trial    # switch to the branch
git pull                                  # update it
git log --oneline -5                      # the last five changes
git status                                # anything changed locally?
```

Remember from 2.3: on the production container, `git checkout` alone changes nothing. The running code comes from the image.

---

3. Test phases

Each phase states its purpose, what must be true before you start, the hardware, numbered steps, the PASS criterion, what to do on failure, and the risk.

Order matters. P1 before everything. P2 before P3 and P4. P8 cannot be done at all yet.

Phases P2, P3 and P4 are covered step by step, with exact expected `STATE` lines, in `docs/BENCH_TEST_PHASE_B.md`. **That document is the authority for those three** — this one gives the purpose, the prerequisites and the gates around them. Do not transcribe its tables; fill them in.

P1 — Power and network

Purpose: establish that every device is powered and reachable before any software is blamed for anything.

Prerequisites: none. This is first.

Hardware: Jetson, switch, both radios, sonar, a laptop with Ethernet.

Steps

1. Power the Jetson and the switch. Confirm link lights on every connected port.

2. Find out what the network **actually is** before testing what it should be. On 2026-10-28 the Jetson's wired side was on `192.168.88.10/24`, the MikroTik factory subnet, not on the plan. From a laptop on the same wired segment:

   ```
   sudo nmap -sn 192.168.88.0/24     # what is really on the switch's subnet
   ping -c 3 192.168.88.10           # the Jetson, as seen on 2026-10-28
   ```

   Only then try the plan, and expect it to fail until somebody has built it:

   ```
   ping -c 3 10.10.20.2    # switch
   ping -c 3 10.10.20.3    # boat radio
   ping -c 3 10.10.20.4    # shore radio
   ping -c 3 10.10.20.10   # Jetson
   ```

   Four failures here is the expected result today, and recording that is itself the finding. Do not start re-addressing anything to make them pass: the plan collides with the sonar (section 2.1) and the decision is open.

3. SSH into the Jetson: `ssh node@node2026` (section 2.2). If the name does not resolve, try `ssh node@192.168.88.10`, the wired address seen on 2026-10-28. Then find out what addresses it actually has today, and write them on the results sheet:

   ```
   ip addr show            # every interface and its address
   ip route                # which one is the default route
   ```

4. On the Jetson, add the temporary address for the sonar's subnet (section 2.1), then:

```
ping -c 3 192.168.2.21
sudo nmap -p- 192.168.2.21
```

Expect exactly `22/tcp open ssh`, `80/tcp open http`, `62312/tcp open`.

5. Record the radio link: signal strength and rate from the MikroTik web interfaces, at the distance you will actually operate at.

PASS: you can say, in writing, what address every device is on today; the sonar shows exactly those three open ports; SSH works; the link holds at operating distance. **PASS does not require the `10.10.20.x` plan to exist** — it requires you to know what does.

FAIL: one device silent → cable, power, and whether your laptop has an address on the same subnet (`ip addr show`). The sonar silent → see P6 and `docs/SONAR_BRINGUP.md` §2. Ports other than those three on the sonar → the firmware is not the one tested on 2026-10-28; record it.

Risk: none beyond normal electrical care.

P2 — Pico: build, flash, and confirm the firmware

Purpose: get `pico-node_v5` onto the Pico and prove it is there. **This gates the entire motor path:** `pico_bridge` forwards motor commands and `MODE AUTO` only to a Pico reporting `ver=5`, and the pre-flight's Pico firmware version check is critical.

Prerequisites: P1. ESC battery **disconnected**.

Hardware: Pico 2, USB cable, a laptop with the Arduino IDE. **The Pico is unplugged from the Jetson for this**, so the stack does not need stopping.

What is believed to be on the board today: v4 — declared by Auxence, never checked against the hardware. If that is right, the stack will refuse to drive it (`pico_bridge` sends only `PING` and `MODE MANUAL` to anything but a v5, and the pre-flight fails), so this phase is not optional before any motor work. Step 10 settles it either way.

This is the first time `pico-node_v5` will have been compiled by the Arduino toolchain at all (TESTED IN SOFTWARE ONLY until now: the host tests compile it against a stub, and all pass). If `Verify` fails, **keep the full error text** — that is a real finding, not your mistake.

Steps — the authority is `docs/BENCH_TEST_PHASE_B.md` §0. In summary:

1. Arduino IDE 2. Add the board URL `https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json` under Preferences → Additional boards manager URLs.
2. Boards Manager → **Raspberry Pi Pico/RP2040/RP2350** (Earle F. Philhower), **version 4.0.1 or newer**. This matters: 4.0.0 is the first release with RP2350 (Pico 2) support and 4.0.1 fixed its tool downloads; a 3.x core does not list the Pico 2 at all.
3. Tools → Board → **Raspberry Pi Pico 2** (RP2350). An RP2040 board is **Raspberry Pi Pico**.
4. Tools → USB Stack → **Adafruit TinyUSB**. The sketch requires it.
5. Open `firmware/pico-node_v5/pico-node_v5.ino`. **Verify** (✓) first.
6. **Upload**. If no port appears: unplug, hold **BOOTSEL** while plugging in, upload again.
7. Serial Monitor, 115200 baud. First line must be `[VER] pico-node 5`, then `STATE ver=5 … rclatch=1 … txdrop=0`.
8. Plug the Pico back into the Jetson's USB hub.
9. Install the udev rules so the port has a stable name (`deploy/udev/README.md`) — plug the Pico in **alone** first to identify it:

   ```
   ls -l /dev/ttyACM*
   udevadm info --query=property --name=/dev/ttyACM0 | grep -E '^(ID_VENDOR_ID|ID_MODEL_ID|ID_SERIAL_SHORT)='
   ```

   `ID_VENDOR_ID=2e8a` is the Pico. Put its `ID_SERIAL_SHORT` in place of `CHANGE_ME_PICO_SERIAL` in `deploy/udev/99-asket.rules`, then:

   ```
   sudo cp deploy/udev/99-asket.rules /etc/udev/rules.d/99-asket.rules
   sudo udevadm control --reload-rules
   sudo udevadm trigger --subsystem-match=tty --action=add
   ls -l /dev/pico                              # -> ttyACMx
   ```

10. With the stack running, confirm from inside the container:

    ```
    ros2 topic echo /pico/status --field data    # expect STATE ver=5 ...
    ```

PASS: `[VER] pico-node 5` on the serial monitor; `/dev/pico` exists and points at the right device; `/pico/status` shows `STATE ver=5`; the bridge log says `Pico firmware : v5, as expected`.

FAIL: compile error → keep the text, it is the first real build. No serial port → BOOTSEL. `ver=` other than 5, or no `ver=` at all → you flashed the wrong sketch, or an older firmware is still on the board. `/dev/pico` missing → the udev serial number is wrong; `udevadm info` again.

Why `rclatch=1` at power-up is correct, not a fault: v5 starts with the RC-loss latch set, so a boat switched on with the arm switch already high does not arm. Cycle Ch7 low then high to clear it. v4 armed in that situation; that was the bug.

Risk: low. ESC battery is disconnected. The main risk is flashing the wrong sketch and not noticing — which is exactly what the version announcement exists to prevent.

P3 — RC receiver and transmitter (SBUS), and failsafe

Purpose: confirm the RP4TD really outputs SBUS, that all four channels decode, and — the important part — **what happens when the radio link is lost.** This is the sovereign safety path.

Prerequisites: P2 passed. ESC battery **disconnected**.

Hardware: Pico (flashed), RP4TD receiver on GPIO5, MT12 transmitter.

How to watch. Everything in this phase is read off the Pico's own status line. In one terminal, inside the container:

```
ros2 topic echo /pico/status --field data
```

A line looks like
`STATE ver=5 mode=2 armed=1 relay=1 rclatch=0 sbusfs=0 …`. The fields that matter here: `mode` (1 = E-stop, 2 = MANUAL, 3 = AUTONOMOUS), `armed`, `relay` (1 closed, 0 open), `rclatch` (1 = an RC loss is latched) and the `sbus*` counters.

Requesting AUTONOMOUS without a Pixhawk. The pre-flight will be NO-GO (no GNSS, no heading), so the GUI refuses Autonomous. Either use the GUI's force path (section 2 of the guide, and P9 step 6), or go round it from the command line, which is an aid and not a safety:

```
ros2 topic pub --once -w 1 /pico/mode_request std_msgs/msg/String "{data: AUTO}"
ros2 topic pub --once -w 1 /pico/mode_request std_msgs/msg/String "{data: MANUAL}"
```

`-w 1` waits for `pico_bridge` to subscribe, so the single message is not lost. Either way **the Pico still decides**: AUTONOMOUS needs Ch8 high, armed, and the latch clear.

Steps. Condensed from `docs/BENCH_TEST_PHASE_B.md` §1, which keeps the full table and is the authority if the two differ.

#	Do	Expect
3.1	Power the Pico with **Ch7 already high** and Ch8 down, then Ch8 to the middle	`armed=0 rclatch=1`, red light. **It must not arm.** (v4 armed here; that was the bug)
3.2	Ch7 low, then high	`rclatch=0`, then `armed=1 mode=2 relay=1`, yellow light, 2 beeps
3.3	Move Ch3 (throttle) and Ch4 (yaw)	both decode and change in the status line
3.4	Ch8 low	`mode=1 armed=0 relay=0`, red light, long beep
3.5	Ch8 high, Ch7 cycled, then request AUTO (box above)	`mode=3`, green light
3.6	**Receiver failsafe:** still in AUTONOMOUS, switch the **transmitter off**	in **under 0.5 s**: `relay=0`, red light, `mode=1 armed=0 rclatch=1 wantauto=0`. Note `sbusfs=` — 1 means the RP4TD reports failsafe; 0 means it simply stopped sending, which is also covered
3.7	**No auto re-arm:** transmitter back on, **Ch7 still high**, wait 10 s	stays `armed=0 rclatch=1 relay=0`, red light
3.8	Request AUTO while latched	refused — the Pico answers `[NAK] MODE AUTO refused`; stays disarmed
3.9	Ch7 low, then high	`rclatch=0`, then `armed=1 mode=2` — **MANUAL, not AUTONOMOUS**
3.10	**SBUS silent:** back to AUTONOMOUS, then unplug the receiver's SBUS lead	same as 3.6 (`sbusfs` irrelevant: no frames at all). Re-plug and repeat 3.7–3.9
3.11	**Serial never blocks:** stop the stack, then hold the port open without reading it — `sudo sh -c 'sleep 60 < /dev/pico'`	during those 60 s the RC still works: Ch8 low turns the light red at once, Ch7 low→high re-arms. Watch the **lights**; nothing is reading `STATE`
3.12	Restart the stack after that	`STATE … txdrop=N` with N > 0 — the lines dropped while nobody read — and everything else normal
3.13	Unplug the Pico's USB for 5 s with the stack running, then re-plug	bridge log: port lost, then within ~2 s `Pico connecte`, `Pico firmware : v5`

PASS: all four channels decode; 3.1 does **not** arm; both kinds of radio loss stop the boat in under 0.5 s in **every** mode including AUTONOMOUS; it does not re-arm by itself; re-arming lands in MANUAL; the Pico keeps running with the serial port blocked.

FAIL: no channels decode → the receiver is not in SBUS mode, or the signal is not inverted as the firmware expects, or GPIO5 is not the wired pin. Channels frozen at their last value → frames are being rejected; look at `sbusok=`/`sbusbad=` in the `STATE` line. Re-arms by itself → **stop**, that is the v4 defect and means v5 is not actually running.

Risk: low with the ESC battery disconnected. **Do not do this phase with the ESCs powered.**

P4 — ESC and motors on a bench supply

Purpose: confirm the motors turn the right way, respond to both the transmitter and autonomous commands, and stop when they should.

> **Prerequisites — read `docs/BENCH_TEST_PHASE_B.md` §2 before this phase.** It requires two things that are **NOT WIRED TODAY**: the relay on GPIO21, and a physical E-stop button in the ESC power line. Without the relay the Pico cannot cut ESC power at all, so a wrong throttle sign or a stuck command spins the thrusters until somebody pulls the battery.
>
> **Neither is fitted today** (the relay: VERIFIED absent; the button: planned, confirmed by Auxence 2026-11-10), **so this phase cannot be done as written.** Wire and test both first. The alternative is testing with the battery connector as your only stop, which is not acceptable with hands near a propeller.

Prerequisites: P2 and P3 passed. Relay and E-stop button wired and tested (press it, ESC rail reads 0 V). Props **out of the water**, see 1.5. Bench supply at the limits in 1.4.

Order: steps 4.1–4.3 **on the bench supply first**. If all three pass, switch off, then connect the LiPo (pack voltage ≥ 14 V first, E-stop pressed in while you connect, then released) and repeat 4.1–4.9.

Steps. Condensed from `docs/BENCH_TEST_PHASE_B.md` §2, which keeps the full table and is the authority if the two differ.

#	Do	Expect
4.1	Arm in MANUAL (Ch8 middle, Ch7 low→high)	`relay=1`, the relay **clicks closed**, ESC start-up tones, and **no spin for 2 s** — the arm window
4.2	A little forward throttle, then yaw	both thrusters forward; yaw spins them opposite
4.3	Ch8 low **while they are spinning**	they stop at once, relay opens, ESC power-down tones

#	Do	Expect
4.4	AUTONOMOUS (Ch8 high, Ch7 cycled, request AUTO as in P3), then publish a small effort: `ros2 topic pub -r 10 /control/effort geometry_msgs/msg/Twist "{linear: {x: 0.15}}"`	both spin slowly forward
4.5	Stop that `ros2 topic pub`	neutral within ~0.5 s — the bridge sends `PING` after 0.3 s, the Pico goes neutral after 0.5 s. **Relay stays closed**
4.6	Restart 4.4, then **transmitter off**	thrusters stop in under 0.5 s, relay opens
4.7	Transmitter back on, Ch7 still high	nothing spins, even with `/control/effort` still being published
4.8	Ch7 low→high	MANUAL; the sticks work; autonomy only after a **new** AUTO request
4.9	**Jetson freezes** while autonomous (4.4 running): `sudo pkill -STOP -f pico_bridge`	neutral within 0.5 s; autonomy withdrawn (`link=0`, MANUAL) after 0.6 s; the transmitter still drives in MANUAL. Then `sudo pkill -CONT -f pico_bridge`

Where the timings come from. Every figure above is a constant in `firmware/pico-node_v5/pico-node_v5.ino`, so you can check them rather than trust this table: `ESC_ARM_DELAY_MS = 2000` (the 2 s no-spin window), `SERIAL_FAILSAFE_TIMEOUT_MS = 500` (neutral after the commands stop), `HEARTBEAT_TIMEOUT_MS = 600` (autonomy withdrawn), and `SBUS_FAILSAFE_TIMEOUT_MS = 300` (radio loss). The 0.3 s before the bridge starts sending `PING` is `cmd_timeout` in `src/control/control/pico_bridge.py`. Mode numbers in the status line: `1` E-stop, `2` MANUAL, `3` AUTONOMOUS.

PASS: every row as written, and **nothing spins at any moment the table does not say it should**.

FAIL: thrusters turn the wrong way → `thr_invert`/`yaw_invert` in `pico_bridge`, and `THR_INVERT`/`YAW_INVERT` in the firmware. Supply current-limits and the ESC browns out → too much throttle (1.4). No click and no `relay=1` → the relay is not actually wired, and you should not be in this phase. **Anything spins when it should not → stop the phase and report it.**

Risk: highest of any bench phase. Spinning propellers, and the dry-run limit on the T200s. Short bursts, low throttle, hand on the supply switch.

P5 — GNSS and the Pixhawk

Purpose: find out which firmware the Pixhawk runs, get position and heading into the stack, and confirm the dual-antenna heading.

Prerequisites: P1. ESC battery disconnected is fine; this phase does not need motors.

Hardware: Pixhawk 4, UM982 GNSS with both antennas, a laptop with QGroundControl, **sky view** (this will not work indoors).

Steps

1. **Which firmware?** This is UNKNOWN today and everything else in the phase depends on it. Three ways, any one is enough:
   - On the Jetson, before launching: `udevadm info` on the Pixhawk's USB device. `26ac:0032` is PX4; `1209:5740` or `1209:5741` is ArduPilot.
   - QGroundControl: Vehicle Setup → Summary names it. Or Analyze Tools → MAVLink Inspector → HEARTBEAT → autopilot: **3 = ArduPilot, 12 = PX4**.
   - Running: `ros2 topic echo /mavros/state --once`. ArduPilot Rover shows modes `MANUAL/HOLD/GUIDED`; PX4 shows `MANUAL/POSCTL/OFFBOARD`.
2. Launch with the matching argument — the default is `ardupilot`:

```
ros2 launch bringup njord.launch.py autopilot:=ardupilot   # or px4
```

3. Confirm the pre-flight's **Flight controller firmware** row passes. It reads the Pixhawk's own heartbeat and **fails if it is the other one**, so a mismatch here is a real finding and not a nuisance.
4. Check each topic the stack needs is actually carrying data — a MAVROS publisher existing is not the same as the Pixhawk sending anything:

```
ros2 topic hz /mavros/global_position/raw/fix
ros2 topic hz /mavros/imu/data
ros2 topic hz /mavros/gpsstatus/gps1/raw
ros2 topic hz /mavros/time_reference
ros2 topic echo /mavros/home_position/home --once
```

5. Set the autopilot parameters for the UM982's dual-antenna heading. The full tables, per firmware, are in `docs/HARDWARE_DECISIONS.md` §4d. **None of it has been tried on this Pixhawk.** The headline items:
   - **ArduPilot:** `SERIAL3_PROTOCOL=5`, `SERIAL3_BAUD=230`, `GPS1_TYPE=25` (UnicoreMovingBaseline — 24 is position only, no yaw),

`GPS1_MB_TYPE=1`, the measured `GPS1_MB_OFS_X/Y/Z`, `EK3_SRC1_YAW=2`, `COMPASS_USE*=0`. Antennas **at least 30 cm apart**.
    - **PX4:** `GPS_1_CONFIG`, `SER_GPS1_BAUD=230400`, `GPS_1_PROTOCOL=6` (NMEA), `EKF2_GPS_CTRL` with bit 3 set (15 works), `SYS_HAS_MAG=0`.
6. **Check the heading physically.** Point the boat at a distant landmark of known bearing. QGroundControl's heading must match, and must follow when you turn the boat. Then confirm it reaches the stack:

```
ros2 topic echo /odometry/filtered
```

PASS: the firmware is identified and the pre-flight agrees; position, heading, attitude, satellites and a GPS time reference all arrive; the heading matches a known bearing and follows a turn.

FAIL: topics silent on ArduPilot → raise the `SR0_*` stream rates (the launch also requests 10 Hz at start-up, 10 s in, but the parameters make it stick). Topics silent on PX4 → `set_stream_rate` is deliberately not sent (PX4 ignores it); USB should stream everything in config mode, and if the link is ever moved to a TELEM port you must set that instance's `MAV_x_MODE` to Onboard. Heading wrong by a constant → antenna offsets or the yaw offset parameter. Heading drifts → a compass is still in the solution; `COMPASS_USE*` or `SYS_HAS_MAG`.

Note: `SYS_HAS_MAG=0` on PX4 is marked **to verify** in the repository — on some versions the EKF then refuses to arm or keeps a compass anyway.

RTK fixed/float is not implemented. Corrections never reach the GNSS, so the best quality the GUI can show is GPS only. Do not record that as a failure.

Risk: none electrical. The risk is a plausible-looking wrong heading, which is why step 6 is physical and not a screenshot.

P6 — Sonar (Omniscan 3D)

Purpose: repeat on the Jetson what was verified on a laptop on 2026-11-06, and then answer the questions that bucket test could not.

Prerequisites: P1. A container of water, or the boat in water.

Hardware: Omniscan 3D, Ethernet, a host with an address on the sonar's subnet.

> **Safety, before anything else.** The transducer goes **under water before it is told to ping**, and is **told to stop before it comes out** (1.6). Pinging in air can damage it. Closing the software does **not** stop it.

Steps — the authority is `docs/SONAR_BRINGUP.md`, which has the full procedure and the exact expected output.

1. Address and ports: section 2.1 of this document, then P1 step 4.
2. **Close SonarView and anything else on TCP 62312.** VERIFIED 2026-11-06: the port takes **one client at a time**. This is the single most common reason the bridge will not connect.
3. Probe it without ROS, the container or the GUI:

   ```
   python3 scripts/sonar_probe.py
   ```

   It defaults to 192.168.2.21; `--host` or `ASKET_SONAR_HOST` for anything else.
4. With the transducer **under water**, start the bridge and RViz. This launch file never starts pinging:

   ```
   source /opt/ros/jazzy/setup.bash && source install/setup.bash
   ros2 launch omniscan_bridge sonar_rviz.launch.py
   ```
5. In a second terminal, once somebody has **checked the transducer is covered**:

   ```
   ros2 service call /omniscan_bridge/start_pinging std_srvs/srv/Trigger
   ```
6. Read the health numbers properly:

   ```
   ros2 topic echo /diagnostics --once
   ros2 topic echo /sonar/status --once
   ```

   Healthy, from 75 s on 2026-11-06: `actual_ping_rate_hz` 5.00, `packet_loss_ratio` 0, `rx_chunks_dropped` 0, `bytes_discarded` 0. `/diagnostics` will still read error level until the sonar's clock is set — **that is the unset clock, not a data fault.**

7. **Stop it before it leaves the water**, while the bridge is still running:

   ```
   ros2 service call /omniscan_bridge/stop_pinging std_srvs/srv/Trigger
   ```

   Watch the points stop. If they do not, **cut its power**.

8. The open questions this phase can close, if you have real water under the boat rather than a bucket:

   - **Are the points in the right place?** Put a known target at a measured range and compare. UNKNOWN today.
   - **Does `pt_type` ever report bottom?** Everything in the bucket was unclassified, which means the boat's seabed-only filter publishes nothing. If real water is the same, that is a finding that affects the whole survey.
   - **Does SonarView hold the port the way a plain client does?** Procedure in `docs/SONAR_BRINGUP.md` §5 "Both at once?" — both connection orders, four named outcomes.

9. **SonarView, if you need Cerulean's own application** — for its live view, its settings or its recordings. It runs as a container on the Jetson, with host networking because its device discovery is a broadcast:

   ```
   sudo podman run -d --name sonarview --network=host \
     -v /usr/SonarView:/userdata:Z \
     -e ENABLE_OS3D=true \
     docker.io/nicknothom/sonarview:latest
   ```

   Then open it **from a laptop**, not on the Jetson:

   ```
   http://<jetson address>:7077
   ```

   `http://localhost:7077` works only if you installed it on the laptop itself. On the Jetson's own browser it fails: `Error creating webgl context`, VERIFIED 2026-10-28.

   Four things about that command:

   - **`sudo`.** Rootless Podman handles `--network=host` differently and discovery broadcasts will not work.
   - **`ENABLE_OS3D=true`** or there is no 3D view. Getting this wrong is the likeliest reason the 3D display is missing.

- **No `--restart` policy.** Do not add `--restart=unless-stopped`: it brings SonarView back automatically after a reboot or a crash, when nobody is watching and the transducer may be in air — and it will take the sonar's single TCP connection, so `omniscan_bridge` then cannot have it. Start it deliberately, each time.
- **Never start pinging out of the water.** The same rule as 1.6, and SonarView will happily do it if you press the button.

Stop it with `sudo podman stop sonarview`, and remember that stopping it does **not** stop the sonar pinging — step 7 does.

10. The sonar's clock: point its NTP at the Jetson's GPS-disciplined server. `docs/SETUP.md` step 0. **This is the most expensive step to get wrong in the whole project** — its default is an internet host, there is no internet in the field, and a wrong sonar clock makes every mission un-georeferenceable after the fact.

PASS: point sets decode with no length mismatches; the ping rate reads 5.00 Hz; `stop_pinging` demonstrably stops it; the clock check passes once NTP is set.

FAIL: will not connect → something else holds the port (step 2). Connects and sends nothing → dry transducer, or it has not been told to ping. Bytes arriving that do not parse → capture them and bring the file back, do not guess at a layout:

```
sudo tcpdump -i eth0 -n -w sonar.pcap host 192.168.2.21 and port 62312
```

Risk: damaging the transducer by pinging in air. That is the whole of 1.6.

P7 — Camera, light tower, BMS

Purpose: the remaining sensors and indicators.

Prerequisites: P1 for the camera, P2 for the lights.

Camera (OAK-D-LR). No driver is installed and the topic name the GUI expects, `/oak/rgb/image_raw/compressed`, is UNKNOWN on real hardware. So this phase begins with installing the driver — see `docs/CAMERA_OAK.md`, which is explicit that nothing in it has run against the real camera. Then:

```
ros2 launch bringup njord.launch.py enable_oak:=true
ros2 topic list | grep -i oak          # what is the name REALLY?
ros2 topic hz <that topic>
```

If the real name differs, that is a one-line change in `src/gui_backend/config/topics.yaml` (`camera_image`), not a code change. **Record the real name.** With the defaults the Camera panel says "Camera not connected", which is true.

Light tower. Three lights on GPIO10/11/12 and a buzzer on GPIO13, driven by the firmware. The convention is checked by the host tests (TESTED IN SOFTWARE ONLY) and is visible throughout P3: red = disarmed or E-stop, yellow = MANUAL armed, green = AUTONOMOUS. Confirm each physically lights, and that the buzzer sounds (2 beeps on arming, a long beep on E-stop). Light codes: `docs/gui/` and the firmware README.

BMS. Only **one** BMS is wired, so `enable_bms2` is false by default. The state of charge is **estimated from cell voltage** (`asket_common/battery.py`), not measured by counting charge in and out (coulomb counting), and reads as "not sent" rather than 0 when unknown.

```
ls -l /dev/serial/by-id/                    # count the MCP2221 devices
ros2 launch bringup njord.launch.py enable_bms:=true
ros2 topic echo /battery/pack1 --once
ros2 topic echo /battery/state --once
```

Compare the reported pack voltage against a multimeter. The udev serial numbers for the MCP2221s are often blank or identical between boards — if so, the rules must key on the USB port path instead (`deploy/udev/README.md`).

PASS: a camera image arrives at a known topic name; all three lights and the buzzer work and follow the modes; pack voltage matches a multimeter within measurement error.

Risk: low.

P8 — Emergency stop chain

Purpose: prove the boat can be stopped by hardware, independently of all software.

This phase cannot be done today

Two parts of the chain are **NOT WIRED** (VERIFIED absent, from `docs/HARDWARE_DECISIONS.md` §3):

- **ESC relay on GPIO21.** Without it the Pico **cannot cut ESC power at all.** The firmware sets the pin and reports `relay=` in its status line, and in P3 you watch that field change — but nothing is connected to it.
- **E-stop button feedback on GPIO20.** The firmware path is compiled out (`ESTOP_FEEDBACK_ENABLED 0`). The GUI and the pre-flight correctly report it as "not sent" / a grey note rather than as safe.

A physical E-stop button in the ESC power line is a separate thing from the GPIO20 feedback: the button cuts the battery to the ESCs with no software involved, and the feedback merely lets the Pico know it has been pressed. **That button is PLANNED, not fitted** (confirmed by Auxence, 2026-11-10).

So three hardware jobs stand between today and a powered motor test: fit the button, wire the relay to GPIO21, and wire the GPIO20 divider. The first two gate P4; the third only gates this phase's step 3 onwards.

When it is wired, the phase is:

1. Measure the relay's output with a multimeter while running P3's steps. `relay=0` in the status line must correspond to the ESC rail actually being open.
2. Press the physical E-stop button with the ESCs powered. The rail must read **0 V**, with no software involved.
3. Enable the feedback: `#define ESTOP_FEEDBACK_ENABLED 1` in the sketch **and** `ESTOP_FEEDBACK_ENABLED = True` in `src/asket_common/asket_common/mode_arbitration.py`. A test fails if the two disagree, deliberately — the firmware and the stack must not hold different beliefs about whether the feedback exists.
4. Re-run the host tests (`make -C firmware/test`, which builds a second binary with the path enabled — currently all pass), then confirm on the bench: the rail low for more than 100 ms disarms, latches and

opens the relay; it stays disarmed when the rail returns; it re-arms only after Ch7 is cycled.
5. Confirm the pre-flight's **E-stop power feedback** row changes from a grey note to a real check.

PASS: the button cuts the rail with no software in the path; the relay opens when the firmware says it does; the feedback latches and does not self-clear.

Risk: this is the phase that protects every other phase. Until it passes, treat the battery connector as the only stop and keep hands away from the propellers.

P9 — Integration with the GUI

Purpose: confirm the whole stack and the GUI agree with the hardware. **None of this has ever been run against the real boat.**

Prerequisites: P1, P2, P5 at least. P4 if you want to see mode changes reach the motors.

Steps

1. Build and deploy the image from `integration-pre-sea-trial` (section 2.3), and **verify you are running it** with the check given there.
2. Start the stack and open the GUI from the **shore laptop**, not the Jetson:

   ```
   http://10.10.20.10:8090
   ```

   (The Jetson's own browser may not render the map — see section 5.)
3. **Pre-flight.** Press Run pre-flight. Expect **NO-GO** until everything is present — that is correct, not a fault. Work the red and amber rows: each one says what to do. There are 21 checks; 11 are critical.
4. **Check the GUI agrees with the hardware**, panel by panel:
   - Mode and arming match the Pico's `STATE` line and the lights.
   - Position and heading match reality.
   - Battery matches a multimeter.

    - The sonar panel says `sonar alive` only when the sonar is actually alive.
    - Anything not fitted reads "not fitted" or "not sent", **never 0 or a healthy value**.

5. **Mode commands.** Request MANUAL and AUTONOMOUS. Each needs two clicks. Confirm the GUI shows **pending** and only then **confirmed** — and that "confirmed" appears only when the Pico's own status agrees.

6. **The NO-GO gate.** With a NO-GO pre-flight, request AUTONOMOUS. It must be **refused with a reason on screen**. Then tick I have read the pre-flight and accept the NO-GO and press I know it is NO-GO, force (twice). Confirm the forced press is logged (`asket_overrides.jsonl`, next to the setup file under `/srv/asket/data/setup/`).

7. **Geofence.** Put `geofence.yaml` at `/srv/asket/data/setup/geofence.yaml` (example: `src/bringup/config/geofence.example.yaml`), drawn around **open water only** — not over moorings, jetties or shore. Confirm the dashed geofence line appears on the map around today's area, and that the pre-flight's Geofence row passes. Without a fence the survey profile is **NO-GO**, because Nav2 would then have no area limit at all.

8. **Link loss.** Walk the shore laptop out of range, or pull its network. The GUI must **say it has lost the link**. A screen that freezes on the last frame while looking live is the worst failure this GUI can have. Confirm the link profile degrades (`full` → `reduced` → `minimal`) and that the GUI says which profile it is on and what it is no longer receiving.

9. **Recording.** Start and stop a recording, then confirm it survives a restart: `ls /srv/asket/data/missions`, `sudo systemctl restart asket.service`, reload the page, the mission is still listed.

PASS: every panel agrees with the hardware; absent things read as absent; commands confirm from observed state; the NO-GO gate refuses and the force path works and is logged; link loss is visible; a recording survives a restart.

FAIL: GUI unreachable → is the container running, is port 8090 open, are you on the right subnet. Blank page with a 200 → the frontend was not built into the image (section 5). A panel reading 0 where hardware is absent → **report it, it is a safety-relevant bug.**

Risk: low, unless P4 has passed and the motors are live. If they are, this is a powered test: section 1.3 applies.

P10 — Quay trials, then on the water

Purpose: the last gate before the boat is autonomous in open water.

Prerequisites: P1–P3, P5, P6, P7, P9 passed. P4 passed. **P8 passed** — do not skip this one.

At the quay, moored, propellers in the water

1. Full pre-flight: **GO**.
2. Arm in MANUAL, drive gently against the mooring lines. Confirm direction and yaw.
3. Ch8 low while driving: immediate stop.
4. Transmitter off while driving: immediate stop, no self re-arm.
5. Sonar under water, pinging, point sets arriving, and health green apart from the clock if NTP is not yet set.
6. Shore link holds at the distance you will operate at.

On the water, under manual control first

1. Manual transit. Confirm position, heading and track on the map.
2. Confirm the geofence is drawn where you think it is, and that the boat is **inside it**.
3. Sonar over real depth: do the points look like a seabed? Does `pt_type` report bottom? (That is the open question from P6.)

Then, and only then, autonomous

1. A short waypoint mission **inside** the geofence, boat **in sight**, transmitter **in hand**, Ch8 ready.
2. Confirm the speed cap: Nav2's survey profile limits to 0.8 m/s and 0.6 rad/s whatever is commanded.
3. Confirm a plan outside the fence is refused.

Final GO / NO-GO checklist

Every line must be **yes**. One "no" is a NO-GO.

#	Item	Yes / No
1	Hardware killswitch tested today, cuts propulsion	
2	RC Ch8 low stops the boat, tested today	
3	Transmitter off stops the boat, tested today, no self re-arm	
4	ESC relay wired and opening (P8)	
5	Physical E-stop button in the ESC line, tested	
6	Pico reports `STATE ver=5`	
7	Pre-flight **GO**, with no forced override	
8	Geofence loaded, drawn over open water, boat inside it	
9	Position and heading correct against a known bearing	
10	Battery above the threshold, matches a multimeter	
11	Sonar in water, pinging, points decoding	
12	Sonar clock set from the Jetson's NTP (else the survey is not georeferenceable)	
13	Shore link holds at operating distance	
14	Recording started and verified on disk	
15	A human is watching the boat, with the transmitter in hand	
16	Someone knows how to stop the sonar before it leaves the water	

There is no obstacle detection. Line 15 is not a formality; it is the only obstacle avoidance this boat has.

---

4. Results sheet

Copy this per session. Fill it in **as you go**, not afterwards.

```
ASKET COMMISSIONING — RESULTS

Date:                  _____________________
Tester(s):             _____________________
Location:              _____________________
Branch:                _____________________   (expected: integration-pre-sea-trial)
Commit (git log -1):   _____________________
Image / container:     _____________________   (image tag, or "dev container")
Code verified as new: yes / no                 (section 2.3 check command)

PHASE RESULTS            PASS / FAIL / NOT DONE      Notes
P1  Power and network      _______________    _____________________________
P2  Pico firmware          _______________    _____________________________
P3  RC and failsafe        _______________    _____________________________
P4  ESC and motors         _______________    _____________________________
P5  GNSS and Pixhawk       _______________    _____________________________
P6  Sonar                  _______________    _____________________________
P7  Camera, lights, BMS    _______________    _____________________________
P8  Emergency stop         _______________    _____________________________
P9  GUI integration        _______________    _____________________________
P10 Quay and water         _______________    _____________________________

FACTS ESTABLISHED  (the most valuable part of this sheet)
Jetson SSH user and host:        _____________________
Jetson wired interface name:     _____________________
Pixhawk firmware:                PX4 / ArduPilot, version _______________
Pico firmware reported:          STATE ver= _______
Camera topic, real name:         _____________________
Sonar address used:              _____________________
Sonar points in the right place: yes / no / not measured
pt_type ever reports bottom:     yes / no / not measured
SonarView + bridge together:     both work / second refused / first cut / not tested
Relay wired:                     yes / no
E-stop button fitted:            yes / no
E-stop feedback (GPIO20) wired:  yes / no
BMS boards wired:                1 / 2
Radio link at operating range:   _______ dBm, _______ Mbit/s, at _______ m

ANYTHING THAT SURPRISED YOU  (verbatim error text is better than a summary)
_____________________________________________________________________
_____________________________________________________________________

LABELS CHANGED  (what moved from NOT VERIFIED / UNKNOWN to VERIFIED, and where
you updated the document)
_____________________________________________________________________
```

---

5. Troubleshooting

Symptom	Probable cause	Check
`ros2: command not found`	ROS not sourced in this shell	`source /opt/ros/jazzy/setup.bash`, then the workspace (2.4)
`ros2 topic list` shows only `/parameter_events`, `/rosout`	ROS sourced, workspace not	`source /opt/njord/setup.bash` (production) or `/workspace/install/setup.bash` (dev)
Code changes have no effect after `git checkout`	Production container: code comes from the image	Rebuild the image (2.3). `git checkout` alone changes nothing
Sonar fixes missing although the image is "latest"	`ghcr.io/.../node-ros-2026:latest` is built from `main` (2026-10-22)	Build from the branch, change `IMAGE=` in `asket.service`, run the verify command (2.3)

Symptom	Probable cause	Check
`bms_reader` or `pico_bridge` exits at import, `No module named 'serial'`	pyserial missing from the image	Fixed in the `Containerfile` (`python3-serial`). If you see it, your image predates that fix — rebuild
Every Nav2 node exits at start-up	Known and fixed: `collision_monitor` polygon `points` written as a nested YAML list (Jazzy needs a string), and a `KeepOutFilter` class that does not exist (`KeepoutFilter`)	Both fixed in `nav2_params.yaml` and `nav2_params_survey.yaml`. On an older checkout, look there
`mavros_node` aborts at start-up	A bare `gcs_url:=` is not a valid parameter override	The launch passes the YAML empty string `''` when `enable_gcs_relay:=false`. Do not hand-edit it to empty
MAVROS topics exist but carry nothing (ArduPilot)	Stream rates not set	Raise `SR0_*` (HARDWARE_DECISIONS §4d). The launch also requests 10 Hz 10 s after start
MAVROS topics exist but carry nothing (PX4)	`set_stream_rate` is deliberately not sent; PX4 ignores it	USB should stream in config mode. On a TELEM port, set `MAV_x_MODE` to Onboard
Pre-flight Flight controller firmware fails	The Pixhawk runs the other firmware from the launch argument	`autopilot:=px4` or `:=ardupilot` (P5 step 1)
Sonar: nothing on TCP 62312	Another client holds the port — it takes **one at a time**	Close SonarView and any probe, then retry
Sonar: connects, sends nothing	Transducer in air, or not told to ping	Under water, then `start_pinging`. In air this is the **expected** reading
Sonar: bytes arrive, nothing parses	Not the format we expect	`tcpdump` it and bring the file back; do not guess a layout (P6)
Sonar `/diagnostics` at error level with good data	Its clock is not set — timestamps read as 1970	Expected until the NTP step (`docs/SETUP.md`). Not a data fault
Sonar still pinging after you closed everything	It keeps pinging with no client (VERIFIED)	`stop_pinging` **while the bridge still runs**, else cut its power
GUI: blank page, HTTP 200, right title	The frontend is present as symlinks and `StaticFiles` rejected them	`follow_symlink=True` in `server.py`. Present in this branch
GUI: HTTP 503, `{"error": "frontend not built"}`	`src/gui_backend/gui_backend/static/` is empty — it is gitignored and not in the repository	`cd src/asket_gui && npm install && npm run build` **before** `colcon build`. The image build does this itself
Browser on the Jetson: `Error creating webgl context`	**VERIFIED 2026-10-28** in SonarView, in the Jetson's own browser. Anything needing WebGL will not draw there	Open it from a laptop instead: `http://<jetson address>:7077` for SonarView, `:8090` for the GUI. Whether the GUI's MapLibre map fails the same way is **NOT VERIFIED** — same cause, never tried
GUI: map is a bare grid on any browser	No tiles: no `.mbtiles` file and no internet	`GET /api/tiles/info` says which. See `docs/map_tiles.md`
GUI unreachable on :8090	Container not running, or wrong subnet	`sudo podman ps`, `sudo systemctl status asket.service`, `ip addr show` on your laptop
Pico: re-arms by itself after a radio loss	That is the **v4** defect	You are not running v5. `ros2 topic echo /pico/status --field data`
Pico: does not arm at power-up with Ch7 high	**Correct v5 behaviour** (`rclatch=1`)	Cycle Ch7 low then high

Symptom	Probable cause	Check
`/dev/pico` missing	udev rule's serial number wrong	`udevadm info` and `deploy/udev/README.md`
Two of everything: duplicate topics, two GUIs	`njord.service` and `asket.service` both running	`sudo systemctl disable --now njord.service njord-update.service` (2.3)
Nothing survives a restart	`/data` not mounted from the host	`asket.service` must mount `/srv/asket/data:/data` (`deploy/jetson/README.md`)
ESC tones loop at switch-on, supply in current limit	Capacitor inrush folding back the supply	Raise the limit to 3 A for the power-up only (1.4)
Motors stop mid-test, ESC browns out	Too much throttle for a 5 A supply	Low throttle. A T200 draws over 20 A at full
Thrusters turn the wrong way	Inversion settings	`thr_invert` / `yaw_invert` (`pico_bridge`), `THR_INVERT` / `YAW_INVERT` (firmware)
`pytest`: 4 errors, `ERR_MODULE_NOT_FOUND: react`	`node_modules` missing	`cd src/asket_gui && npm install`. Pre-existing, not your change

---

6. Open questions and pending decisions

These need a person, not a test. Each one blocks or changes something.

Decisions for the team

1. **The IP plan.** The sonar is on `192.168.2.21`; the plan is `10.10.20.0/24`; `.21` in the plan is the OAK-D-LR; and SonarView makes the sonar's two internal boards talk on `192.168.2.x`. Nobody has decided which subnet the sonar ends up on, who gets `.21`, or whether those boards follow a re-address. **Until this is decided the software follows the measurement.** Not a decision to take on a survey day.
2. **The registry token in `scripts/init.sh`.** A credential is hard-coded in that file and therefore in the git history. **It must be revoked and purged from history.** Known to the team; recorded here so it is not forgotten. Not reproduced in this document.
3. **CI only builds `main`.** `.github/workflows/build.yaml` publishes `ghcr.io/node-engineering-club/node-ros-2026:latest` from `main` alone, so the published image never contains branch work. Either merge to `main` before the trial, or accept building on the Jetson every time — and say which, in writing, because somebody will otherwise deploy `:latest` and wonder why the sonar is broken.
4. **Where SonarView runs** (Docker on the Jetson, or a laptop). Open, and it interacts with the one-client-at-a-time limit.

5. **The OAK-D-LR.** No driver installed, no confirmed topic name, and its planned address collides with the sonar's current one.
6. **The LTE beacon** (LilyGO T-SIM7000G) and the **Cloudflare relay hostname**. Both open team decisions.
7. **The physical E-stop button is planned, not fitted** (confirmed by Auxence). Together with the unwired relay on GPIO21 this is what blocks P4: wiring and testing both is the gate, and it is a hardware job somebody has to schedule.

Facts nobody knows yet

#	Question	Blocks
1	~~The Jetson's SSH username and hostname~~	**Answered 2026-10-28:** `node@node2026`
2	Which firmware the Pixhawk runs, PX4 or ArduPilot	P5, and the critical pre-flight check
3	What firmware is on the Pico right now	P2. **v4 is declared** (Auxence), never checked. The stack drives v5 only, so if that is right the motor path is blocked until v5 is flashed
4	Whether `pico-node_v5` compiles under the Arduino toolchain	P2 — it has never been tried
5	Whether the RP4TD is in SBUS mode, and what it does on radio loss	P3, which is the sovereign safety path
6	Whether the sonar's points are in the right place	P6, P10 — the survey's whole value
7	Whether sonar `pt_type` ever reports bottom	P6, P10 — if not, the seabed filter publishes nothing
8	Whether SonarView holds the sonar port like a plain client	P6, and whether the GUI can ever show sonar health during a recording
9	Whether `SYS_HAS_MAG=0` lets a PX4 EKF arm	P5, PX4 only
10	The real OAK-D-LR topic name	P7, P9
11	Whether the MCP2221 serial numbers are unique	P7 — if not, udev must key on the USB port path
12	The camera stream's cost on the Jetson (JPEG encode plus the vision node)	nothing yet; it is why the rate ceiling is 15 Hz, not 30
13	~~Whether the Jetson's browser really lacks WebGL~~	**Answered 2026-10-28:** `Error creating webgl context` in SonarView on the Jetson. Still open for the GUI's map specifically — same cause, never tried. Use a laptop either way
14	The 2198 mono-profile payload layout	`sonar_bridge_node` in `cloud_bridge` assumes one; **do not trust its output**
15	Whether the sonar's two boards' addresses survive a re-address	decision 1

Known-stale reference material

`Mission controls and documents/KELP_MISSIONS.md` and `KELP_CAPABILITIES.md` are the client's original design documents. They still give the Omniscan's default address as `192.168.2.92`, which is **wrong for this unit** — it answers on `192.168.2.21`. They are kept as the historical input they are and have not been edited. Where they disagree with `docs/HARDWARE_DECISIONS.md`, that file wins.

---

7. Glossary

Term	Meaning
ArduPilot / PX4	The two competing open-source autopilot firmwares. Which one this Pixhawk runs is unknown, and the software has to be told.
arm window	The 2 s after the relay closes during which the Pico holds the thrusters at neutral whatever the sticks say, so an ESC that is still booting cannot be handed a throttle. `ESC_ARM_DELAY_MS` in the firmware.
Arming	Putting the vessel in a state where the motors may turn. Requires the RC arm switch (Ch7) and the mode switch (Ch8).
ASV	Autonomous surface vessel. This boat.
BMS	Battery management system. The board that monitors the LiPo's cells.
BOOTSEL	The button on a Pico. Held while plugging it in, the Pico appears as a USB drive you can write firmware to — the way in when no serial port shows up.
Ch7 / Ch8	Radio channels. Ch7 is the arm switch; Ch8 selects E-stop / MANUAL / AUTONOMOUS and **is sovereign** — it stops the boat in firmware.
colcon	The build tool for a ROS 2 workspace. `colcon build` compiles it; `colcon test` runs the ROS-dependent tests.
Container / image	A packaged filesystem with its own installed software. An image is the package; a container is a running instance.
Dev container	A container with your source code mounted into it, so edits take effect without rebuilding. The opposite of the production image.
DHCP	A device being given its network address automatically when it connects, instead of having a fixed one. The Jetson's Wi-Fi uses it, so that address changes — look it up, never write it down.
Diagnostics	`/diagnostics`, the ROS topic where nodes report their own health as key/value pairs. More informative than the GUI.
EKF	Extended Kalman filter. Fuses GNSS and IMU into one position and heading estimate (`/odometry/filtered`).
ESC	Electronic speed controller. Turns a PWM pulse into motor power.
failsafe	What the radio receiver does when it loses the transmitter: either it says so with a flag (`sbusfs=1`) or it simply stops sending. The firmware treats both the same — stop the boat.
Geofence	A polygon the boat may not leave. Nav2 treats outside it as impassable, so no path is planned or followed out of it.
GNSS	Satellite positioning, the general term (GPS is one system).
GO / NO-GO	The pre-flight's verdict. NO-GO blocks autonomous mode unless deliberately forced and logged.
Jetson	The NVIDIA AGX Orin, the boat's main computer. Runs ROS 2 and the GUI, in a container.
Killswitch	The hardware switch that cuts propulsion with no software involved. Sovereign.

Term	Meaning
MAVLink / MAVROS	The protocol autopilots speak, and the ROS 2 node that translates it into topics.
mbtiles	A single-file map-tile database, so the map works with no internet.
Nav2	The ROS 2 navigation stack: plans paths and follows them. Here it knows a geofence and a speed limit, and **nothing about obstacles**.
nmap	A command-line tool that lists which network ports a device has open. Used to prove the sonar's data port is 62312 and nothing else.
Node	One ROS 2 program.
NTP	Clock synchronisation over a network. The sonar's clock must come from the Jetson's GPS-disciplined server, or the survey cannot be georeferenced.
Pico	Raspberry Pi Pico 2 microcontroller. Drives the motors, reads the radio, and owns the safety chain.
Ping Protocol	Blue Robotics' / Cerulean's binary message format, which the sonar speaks. VERIFIED 2026-11-06.
Pixhawk	The flight controller. Provides position, heading and IMU. **It does not drive the motors** — the Pico does.
Podman	A container runtime, command-compatible with Docker.
Pre-flight	The built-in test (`system_test`), 21 checks, 11 critical, producing GO or NO-GO.
pt_type	A field on each sonar point saying what the sonar thinks it is — seabed, surface, unclassified. The boat's filter keeps only seabed points, so if this never says seabed, the filter publishes nothing.
PWM	Pulse-width modulation. 1000–2000 µs pulses, 1500 neutral, from the Pico to the ESCs.
QGroundControl	The desktop application used to configure a Pixhawk: read its firmware, set its parameters, check its heading.
rclatch	A field in the Pico's status line. `1` means an RC-loss latch is held and the boat will not arm until the arm switch is cycled. `1` at power-up is correct.
rclpy	The Python library for ROS 2. Tests that import it only run where ROS is installed, which is why six are skipped here.
Relay (A3KM)	The relay on GPIO21 that would let the Pico cut ESC power. **Not wired.**
RTK	Differential GNSS giving centimetre accuracy, using corrections from a fixed base. **Not implemented** in this software.
SBUS	The serial protocol the RC receiver uses to send all channels on one wire. Inverted; the firmware decodes inverted SBUS only.
scp	Copy files to or from another machine over SSH. Same arguments as `cp` , with `user@host:` in front of the remote side.
SonarView	Cerulean's own application for the sonar: its settings, its live view, its recordings. **Takes the sonar's only TCP connection.**
Sourcing	Running a script that sets environment variables in your current shell, so `ros2` commands work. Per-shell; every new terminal needs it.
SSH	A remote terminal: you type on your laptop, the commands run on the Jetson. `ssh node@node2026` .
STATE line	The status line the Pico prints about ten times a second, read with `ros2 topic echo /pico/status --field data` . Key fields: `ver` (firmware), `mode` (1 E-stop, 2 MANUAL, 3 AUTONOMOUS), `armed` , `relay` , `rclatch` , `wantauto` (the Jetson has asked for autonomy), `txdrop` (status lines dropped because the host stopped reading), and the `sbus*` counters including `sbusfs` (receiver failsafe flag).
systemd / `systemctl`	Linux's service manager. `asket.service` is the unit that keeps the stack running.
T200	Blue Robotics thruster. **Water-lubricated** — only short low-throttle bursts in air.
tcpdump	A command-line tool that records raw network traffic to a file, for somebody else to decode later. Used when the sonar sends bytes we do not understand.

Term	Meaning
TinyUSB	The USB library the Pico firmware uses. It must be selected in the Arduino IDE (Tools → USB Stack) or the sketch will not build.
Topic / Service	A named stream of ROS messages (many to many, no reply) / a request-response call (one answer).
udev	The Linux subsystem that names devices. Gives the stable `/dev/pico`, `/dev/pixhawk`, `/dev/bms_pack1` names instead of `/dev/ttyACM0`, which moves.
UM982	The Holybro dual-antenna GNSS receiver. Two antennas give a true heading with no magnetometer.
WebGL	The browser feature the map needs in order to draw. A browser without it shows no map.

Where to look next

For	Read
The hardware, addresses, and what is verified	`docs/HARDWARE_DECISIONS.md`
The safety rules	`docs/safety.md`
Pico, RC and motor bench steps, in full	`docs/BENCH_TEST_PHASE_B.md`
Firmware versions, what changed, how to flash	`firmware/README.md`
The sonar, in full	`docs/SONAR_BRINGUP.md`
Steps that can only be done on the hardware	`docs/SETUP.md`
Running the stack on the Jetson so data survives	`deploy/jetson/README.md`
Stable device names	`deploy/udev/README.md`
The camera	`docs/CAMERA_OAK.md`
Map tiles	`docs/map_tiles.md`
The one page for the operator on the beach	`docs/OPERATOR_CHECKLIST.md`
Taking over the GUI	`docs/GUI_DEVELOPER_GUIDE.md`
What is left to do	`TODOS.md`